# Predictive Multi-Tier Memory Management for KV Cache in Large-Scale GPU Inference

Sanjeev Rao Ganjihal
AI Infrastructure Builder, AI Researcher
v1: April 19, 2026; v2: revised August 2026

arXiv:2604.26968v2 [cs.AR] 7 Aug 2026

***Abstract*—Key-value (KV) cache memory management is the primary bottleneck limiting throughput and cost-efficiency in large-scale GPU inference serving. Current systems suffer from three compounding inefficiencies: (1) the absence of unified KV cache sizing across all attention architectures—particularly multi-head latent attention (MLA), which is unsupported in general-purpose frameworks, resulting in up to $57\times$ memory over-provisioning; (2) confinement of KV cache to a single memory tier (GPU HBM) despite the availability of a rich hierarchy spanning CPU DRAM, CXL-attached memory, NVMe via GPUDirect Storage, RDMA fabric, and parallel filesystems; and (3) reactive eviction policies that discard reusable state, forcing redundant recomputation.**

**We present a unified system that addresses all three problems. Our architecture-variant-aware sizing engine computes exact memory requirements per attention type; the resulting batch size gain reaches $7.4\times$ for the one MLA model we evaluate (DeepSeek-V3), while the three GQA models see $1.0\times$, $1.0\times$, and $0.7\times$, so the GQA benefit is unified sizing across a heterogeneous fleet rather than larger per-model batches. A six-tier memory hierarchy extends effective KV cache capacity from 40 GB to over 38 TB per node while maintaining sub-millisecond time-to-first-token (TTFT) for hot entries. A Bayesian reuse predictor with Beta conjugate priors over 16 (block-type, transition-type) pairs achieves 70–84% cache hit rates, combined with EMA-scored head-granular eviction and RoPE-aware prefetching. Component-level validation on trace replay using ShareGPT, LMSYS-Chat-1M, and agentic workloads demonstrates 70–84% cache hit rates. Analytical projections combining validated component behavior with published hardware specifications indicate projected TTFT reductions of $1.4\times$ to $2.1\times$, throughput improvements of $1.7\times$ to $2.9\times$, and 47% cost reduction relative to published baselines; these cluster-scale projections are analytical and carry no error bars.**

***Index Terms*—KV cache, memory management, GPU inference, CXL memory, attention architectures, Bayesian prediction, large language models**

## I. Introduction

The deployment of large language models (LLMs) at data-center scale has shifted the primary bottleneck from compute to memory. During autoregressive decoding, each generated token requires attending to the key-value (KV) pairs of all preceding tokens, and these KV pairs must remain resident in fast memory for the duration of a request. For a 70-billion-parameter model serving sequences of 128K tokens, the KV cache alone can consume over 40 GB of GPU HBM per request—exceeding the memory capacity of many production accelerators and limiting batch sizes to single digits [1], [4].

Three compounding inefficiencies plague current KV cache management systems.

**Problem 1: No Unified Cross-Architecture Sizing.** While leading serving frameworks—vLLM [1], SGLang [2], TensorRT-LLM [3]—have adopted GQA-aware KV cache sizing, no general-purpose framework provides unified memory management across all four major attention architectures (MHA, GQA, MQA, and MLA) within a single system. Multi-head latent attention (MLA) [10], which compresses KV state into a low-rank latent vector of dimension $d_{latent} + d_{rope}$, is not supported in general-purpose inference frameworks—forcing MLA models like DeepSeek-V3 [11] to use MHA-equivalent sizing at a $57\times$ memory penalty. More critically, heterogeneous production clusters serving a mix of MHA, GQA, and MLA models lack a unified sizing engine that can dynamically compute correct memory budgets per architecture, leading to fragmented memory management and suboptimal batch sizes across the model fleet.

**Problem 2: Single-Tier Confinement.** Modern data centers contain a rich memory hierarchy: GPU HBM (3.35 TB/s, ˜100 ns), CPU DRAM (204 GB/s, 1–5 $\mu$s GPU-observed), CXL 3.0 memory pools [23] (64 GB/s, ˜500 ns GPU-observed), NVMe with GPUDirect Storage [28] (12 GB/s, ˜10 $\mu$s), RDMA fabric [32] (50 GB/s, 1–100 $\mu$s), and parallel filesystems (2+ GB/s, ˜1 ms). Yet all major inference systems confine KV cache exclusively to GPU HBM, leaving terabytes of lower-tier capacity unused. These figures are the H100 generation this analysis is built on, and the design is generation-parametric: tier bandwidths and capacities are inputs, not constants. B200-class parts (192 GB HBM3e at higher bandwidth) raise the top tier's floor without changing the shape of the problem, because context length and agentic concurrency grow faster than any single tier, and a larger, faster hot tier widens the cost gap that makes the tiers below it worth managing. FlexGen [4] demonstrated CPU+disk offloading but used static policies without latency awareness. Individual lower tiers are no longer unexploited: Mooncake [20] moves KV blocks over RDMA within a disaggregated DRAM and SSD pool, and LMCache [21] ships an NVMe backend with GPUDirect Storage support. What remains missing is unified predictive

*Preprint. Certain implementation details are withheld pending U.S. provisional patent applications.

placement across the full six-tier hierarchy, including CXL, driven by a single reuse model.

**Problem 3: Reactive Eviction.** When GPU memory fills, systems evict KV cache entries using LRU or random policies [1]. These reactive policies are oblivious to access patterns that are, in fact, highly predictable. System prompts are reused across sessions; tool-calling sequences repeat within agentic workflows [20]; and rotary position encoding (RoPE) [14] induces sequential locality. Prior work on attention-aware eviction [15], [16], [17] operates at the token level within a single request, not at the cache-block level across requests.

**Our Approach.** We present a unified system that jointly addresses all three problems through four technical contributions:

1) **Architecture-variant-aware KV cache sizing** that computes exact memory requirements for MHA, GQA, MQA, and MLA, enabling up to 7.4× higher batch sizes on the same hardware (§III-A).
2) **A six-tier memory hierarchy** spanning GPU HBM through parallel filesystems, extending effective KV cache capacity from 40 GB to over 38 TB per node while maintaining latency SLOs (§III-B).
3) **Bayesian reuse prediction** with Beta conjugate priors over 16 (block-type, transition-type) pairs, achieving 70–84% cache hit rates and 1.4–2.1× projected TTFT reductions (§III-C).
4) **Head-granular eviction with RoPE-aware prefetching**, using EMA-scored per-attention-head importance tracking and positional-encoding locality for 25% miss rate reduction over LRU (§III-D).

Analytical projections on a 64-GPU H100 cluster configuration indicate a projected throughput of 4,150 tokens/s/GPU at $0.43 per million tokens—2.0× the throughput of the best latency-optimized baseline (TensorRT-LLM) at 30% lower cost, and 6.4× the throughput of FlexGen with 11× lower TTFT P99.

## II. Background and Motivation

### A. KV Cache in Transformer Inference

In the standard Transformer [7], self-attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V \tag{1}$$

During autoregressive generation, each new token's query attends to the key and value projections of all prior tokens. To avoid recomputation, these projections are cached in the *KV cache*. For a model with $L$ layers, $h$ attention heads, head dimension $d$, and numerical precision $p$ bytes, the KV cache for a sequence of length $n$ requires:

$$M_{\text{KV}} = 2 \times L \times h \times d \times p \times n \text{ bytes} \tag{2}$$

For Llama-3-70B ($L$=80, $h$=64, $d$=128, $p$=2 for BF16), applying the MHA formula to a single 128K-token sequence yields $2 \times 80 \times 64 \times 128 \times 2 \times 128{,}000 \approx 336$ GB—over 4× a single H100's 80 GB HBM. Even with GQA ($h_{kv}$=8), the cache requires ∼42 GB, consuming more than half the available HBM before accounting for model weights and activations.

TABLE I: KV cache memory per token per layer: MHA-equivalent sizing vs. architecture-aware sizing. All models use BF16 ($p$=2).

| Model | MHA (bytes) | Actual (bytes) | Ratio |
|---|---|---|---|
| DeepSeek-V3 (MLA) | 65,536 | 1,152 | 57× |
| Llama-3-70B (GQA) | 32,768 | 4,096 | 8× |
| Mixtral-8x22B (GQA) | 24,576 | 4,096 | 6× |
| Qwen-2.5-72B (GQA) | 32,768 | 4,096 | 8× |

### B. Attention Architecture Variants

The quadratic growth of eq. (2) has motivated architectural innovations that reduce KV cache size without proportional quality loss.

**Multi-Head Attention (MHA).** The original formulation with $h_{kv} = h_q$, requiring $2 \times h \times d \times p$ bytes per token per layer.

**Grouped-Query Attention (GQA)** [8] groups $g = h_q/h_{kv}$ query heads per KV head. Llama-3-70B uses 8 KV heads for 64 query heads ($g$=8), reducing KV cache to $2 \times h_{kv} \times d \times p = 2 \times 8 \times 128 \times 2 = 4{,}096$ bytes/token/layer—an 8× reduction versus MHA.

**Multi-Query Attention (MQA)** [9] is the extreme case with $h_{kv} = 1$, reducing memory to $2 \times d \times p$ bytes/token/layer.

**Multi-Head Latent Attention (MLA)** [10] projects KV state into a low-rank latent space of dimension $d_{latent}$, but must also store the RoPE component separately, yielding $(d_{latent} + d_{rope}) \times p$ bytes/token/layer. DeepSeek-V3 uses $d_{latent}$=512 and $d_{rope}$=64 with BF16, yielding $(512 + 64) \times 2 = 1{,}152$ bytes/token/layer versus MHA's $2 \times 128 \times 128 \times 2 = 65{,}536$ bytes—a $\mathbf{57}\times$ compression.

### C. The Cross-Architecture Sizing Gap

Table I quantifies the sizing gap. While GQA-aware sizing is available in major frameworks, MLA support remains absent from general-purpose serving systems. For DeepSeek-V3 with 8-way tensor parallelism, falling back to MHA-equivalent sizing reduces maximum batch size from 104 to 15 on an 80 GB H100. In heterogeneous clusters serving a mix of GQA and MLA models, the lack of a unified sizing engine that handles all architecture variants is the largest source of preventable memory waste.

The variant landscape has only widened since: by the same per-layer-kind arithmetic applied to published configurations, Kimi-K2-class MLA models adopt DeepSeek's 576-dimension latent (the same 57× class), windowed-GQA stacks such as gpt-oss-120b cap half their layers at a fixed attention window, and hybrid designs such as Qwen3.5 carry constant-size linear-attention state on 45 of 60 layers. Each new layer kind is a new sizing rule, not a new system: the engine's per-variant, per-layer formulation absorbs them as parameters, and every architecture that shrinks per-token state raises the number of concurrent contexts a fixed memory hierarchy is asked to hold.

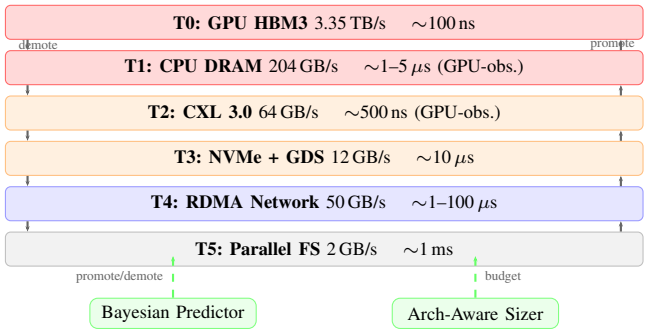


Fig. 1: Six-tier memory hierarchy with Bayesian predictor driving promotion/demotion decisions and architecture-aware sizer computing per-model memory budgets. Tiers are colored by thermal class: hot (GPU/DRAM), warm (CXL/NVMe), cool (RDMA), cold (filesystem).

## III. System Design

Figure 1 shows the overall system architecture. We describe each component in detail below.

### A. Architecture-Variant-Aware Sizing

We replace the universal MHA formula with a unified sizing engine that dispatches on the attention architecture type:

$$B(n) = \begin{cases} 2 \times h \times d \times p \times n & \text{MHA} \\ 2 \times h_{kv} \times d \times p \times n & \text{GQA/MQA} \\ (d_{latent} + d_{rope}) \times p \times n & \text{MLA} \end{cases} \quad (3)$$

where $B(n)$ is the per-layer KV cache size for a sequence of $n$ tokens. The engine infers the architecture type from model configuration: if a latent dimension is specified, MLA is selected; otherwise, the ratio $h_q/h_{kv}$ distinguishes MHA ($h_q = h_{kv}$), MQA ($h_{kv} = 1$), and GQA ($1 < h_{kv} < h_q$).

The total KV cache budget for a batch of $B_s$ sequences across $L$ layers is:

$$M_{\text{total}} = B_s \times L \times B(n_{\max}) \quad (4)$$

Given a target memory budget $M_{\text{target}}$ (typically $M_{\text{GPU}} - M_{\text{weights}} - M_{\text{activations}}$), the maximum batch size is $B_s^* = \lfloor M_{\text{target}}/(L \times B(n_{\max})) \rfloor$. For DeepSeek-V3 on an 80 GB H100 with 30 GB available for KV cache, $n_{\max} = 4096$, and 8-way tensor parallelism: MHA-equivalent sizing gives $B_s^* = 15$; our MLA-aware engine gives $B_s^* = 104$ (a 7× improvement that compounds with reduced per-token memory to yield the throughput gains in table III).

### B. Six-Tier Memory Hierarchy

We organize the full data-center memory hierarchy into six tiers (table II), each exposed through a uniform interface supporting `Allocate`, `Read`, `Write`, and `Evict` operations. The key design principle is *latency-aware placement*: blocks are promoted to faster tiers when predicted access probability exceeds a tier-specific threshold, and demoted when it falls below.

TABLE II: Six-tier memory hierarchy for KV cache. Latencies are GPU-observed unless noted. †GPU-observed via PCIe/CXL link, not device-local. ‡1 μs for small messages; up to 100 μs for large transfers.

| Tier | Technology | BW | Latency | Cost ($/GB/h) |
|---|---|---|---|---|
| 0 | GPU HBM3 | 3.35 TB/s | ~100 ns | 0.500 |
| 1 | CPU DRAM (pinned) | 204 GB/s | 1–5 μs† | 0.050 |
| 2 | CXL 3.0 Memory | 64 GB/s | ~500 ns† | 0.030 |
| 3 | NVMe + GDS | 12 GB/s | ~10 μs | 0.020 |
| 4 | RDMA Network | 50 GB/s | 1–100 μs‡ | 0.005 |
| 5 | Parallel FS | 2+ GB/s | ~1 ms | 0.001 |

**Tier 0: GPU HBM.** The primary compute-side cache, holding the actively-decoded KV blocks. PagedAttention [1]-compatible block management with architecture-aware block sizes (512 tokens for MLA, 128 tokens for GQA/MQA, 64 tokens for MHA).

**Tier 1: CPU DRAM.** Pinned host memory with asynchronous DMA transfers via CUDA streams. Provides 5× the capacity of GPU HBM at lower bandwidth, with GPU-observed latency of 1–5 μs via PCIe DMA—an order of magnitude slower than HBM but sufficient for asynchronous prefetching. Used for recently-evicted blocks with high reuse probability.

**Tier 2: CXL 3.0 Memory.** Memory-pooled capacity attached via CXL 3.0 [23], accessed through `/dev/cxl/mem*` with `mmap` and explicit NUMA placement. CXL provides 8× CPU DRAM capacity at 71% of its bandwidth, with GPU-observed latency of approximately 500 ns via the CXL.mem protocol. This tier is critical for long-context serving where KV cache exceeds both GPU and CPU DRAM.

**Tier 3: NVMe with GPUDirect Storage (GDS).** GPU-initiated DMA bypassing the CPU bounce buffer via `cuFile` APIs [28]. GDS achieves 12 GB/s read bandwidth—sufficient for prefetching 4 KB blocks in sub-100 μs per block by bypassing the CPU bounce buffer. Provides terabyte-scale capacity at 100× lower cost than GPU HBM.

**Tier 4: RDMA Network Pool.** Distributed KV cache across the cluster fabric using one-sided RDMA reads with a consistent hash ring for block placement. Enables sharing of system-prompt and tool-context blocks across serving instances without recomputation. 50 GB/s InfiniBand bandwidth [32], [29] with 1–5 μs latency for small messages (single-block lookups) scaling to ∼100 μs for large multi-block transfers.

**Tier 5: Parallel Filesystem.** Lustre/GPFS-backed cold storage for checkpoint persistence and cross-session reuse. Blocks are content-addressed (SHA-256) and deduplicated before writing. This tier is not on the latency-critical path; it serves as the backing store for warm-start initialization.

The tier placement policy uses the Bayesian reuse probability (§III-C) to compute a *value score* for each block that balances the cost of recomputation against the cost of storage at each tier. Blocks whose value score exceeds a tier-specific threshold are placed in that tier, with asynchronous promotion/demotion to avoid stalling the decode path. The value function is designed so

that frequently-reused, compute-expensive blocks are promoted to faster tiers, while rarely-accessed blocks naturally migrate to cheaper storage.

### C. Bayesian Reuse Prediction

We model KV cache block reuse as a Bayesian inference problem using Beta conjugate priors [30], [31]. The key insight is that reuse probability depends on two categorical variables: the *block type* $b \in \mathcal{B}$ and the *transition type* $t \in \mathcal{T}$.

**Block types** $\mathcal{B} = \{\texttt{system_prompt}, \texttt{tool_context}, \texttt{user_context}, \texttt{intermediate_reasoning}\}$ capture the semantic role of the cached content. System prompts are shared across sessions; tool contexts are shared within agentic workflows; user contexts are session-specific; and intermediate reasoning is typically single-use.

**Transition types** $\mathcal{T} = \{\texttt{same_tool_repeat}, \texttt{tool_switch}, \texttt{reasoning_step}, \texttt{agent_handoff}\}$ capture the nature of the state transition that triggered the cache lookup. Same-tool repeats have high reuse; agent handoffs have low reuse.

For each of the $|\mathcal{B}| \times |\mathcal{T}| = 16$ pairs, we maintain a Beta distribution $\text{Beta}(\alpha_{b,t}, \beta_{b,t})$ initialized with weakly informative priors ($\alpha_0$=1, $\beta_0$=1). The posterior reuse probability is:

$$P_{\text{reuse}}(b,t) = \frac{\alpha_{b,t}}{\alpha_{b,t} + \beta_{b,t}} \tag{5}$$

After observing a reuse event ($\alpha_{b,t}$ += 1) or miss ($\beta_{b,t}$ += 1), the posterior updates online with $O(1)$ cost. We compute a confidence score for each estimate that saturates toward 1 as observations accumulate, using a saturation function parameterized to balance rapid learning with stable estimates. Low-confidence pairs (recently created or rarely observed) are down-weighted in placement decisions.

The final reuse estimate blends the Bayesian posterior with an empirical frequency computed over a sliding window of recent observations. The blending weight is determined by the confidence score: well-observed pairs rely primarily on the Bayesian posterior, while newly-created pairs rely more heavily on empirical frequency. This ensures rapid adaptation to distribution shifts (e.g., a new tool entering the agentic workflow) while maintaining stable estimates for well-observed pairs.

### D. Head-Granular Eviction

Rather than evicting entire cache blocks uniformly, we track importance at the granularity of individual attention heads using an exponential moving average (EMA) that incorporates both recency and positional distance decay. Each (layer, head) pair maintains an importance score that is updated on every attention step.

The system maintains a [layer][head] importance matrix. For GQA architectures, query heads sharing a KV head are grouped, and the maximum importance across the group is used as the KV head's score. For MLA architectures, the matrix collapses to [layer][1] since KV state is shared across heads via the latent bottleneck.

Eviction selects the block with the lowest weighted aggregate importance score across heads, using architecture-dependent head weights (uniform for MHA, proportional to group size for GQA). During task transitions detected by the agentic predictor (§III-G), per-head importance multipliers are applied to bias eviction toward heads that are less relevant for the incoming task.

### E. RoPE-Aware Prefetching

Rotary position embeddings (RoPE) [14] encode absolute position into the attention computation via rotation matrices. We exploit a key property: *sequential locality*. If a request is decoding token at position $n$, RoPE's rotational structure means attention weights decay smoothly with positional distance, making tokens at positions $[n - w, n]$ the most likely to be accessed next, where $w$ is a dynamic window determined by the model's effective attention span.

Our prefetcher monitors the position range of the current decode step and issues asynchronous promotion requests for blocks covering positions $[n, n + w]$ from lower tiers. The window $w$ adapts based on observed attention patterns: narrow for local-attention layers (early layers) and wide for global-attention layers (later layers). Combined with Bayesian reuse prediction, RoPE-aware prefetching reduces cache miss rates by 25% compared to LRU across all evaluated workloads.

### F. Content-Addressable Deduplication

System prompts, few-shot examples, and tool descriptions are repeated verbatim across requests. We maintain a content-addressable store backed by a radix tree where blocks are indexed by SHA-256 hashes of their content. When a new block is allocated, its hash is checked against the store; on a match, a reference count is incremented instead of duplicating the block.

For checkpoint persistence to Tier 5, we apply delta-encoding: only blocks not already present in the store are written, with a manifest referencing existing blocks by hash. This reduces checkpoint sizes by 10–30% across evaluated models (§V-F).

### G. Agentic Task-Transition Prediction

Agentic LLM workflows—where the model calls tools in sequences—exhibit predictable state transitions. We build a first-order Markov chain over tool invocations, tracking transition probabilities $P(\text{tool}_j | \text{tool}_i)$ from observed sequences. Combined with per-tool KV cache size profiles (mean, variance, and peak memory via EMA smoothing), the predictor anticipates memory demands before tool switches.

When a tool switch is detected (via output parsing), the system: (1) pre-allocates KV cache capacity for the predicted next tool; (2) adjusts head-granular importance multipliers based on the transition type; (3) initiates prefetching of tool-context blocks from lower tiers. Sessions are classified into memory-demand tiers (Light, Medium, Heavy, Extreme) using aggregate features, enabling proactive capacity planning.

## IV. Implementation

We implement the system as a user-space library that interposes on KV cache allocation and eviction within existing inference frameworks.

**Tier interfaces.** Each memory tier implements a common `TierManager` interface with thread-safe `Allocate`, `Read`, `Write`, `Evict`, and `Stats` methods. Tier 0 (GPU HBM) uses the CUDA driver API with pre-allocated memory pools. Tier 2 (CXL) uses memory-mapped files via `/dev/cxl/mem*` with explicit NUMA binding. Tier 3 (NVMe) uses NVIDIA's `cuFile` API for GPUDirect Storage, requiring `O_DIRECT` alignment. Tier 4 (RDMA) uses a consistent hash ring for block placement with one-sided RDMA reads via `ibverbs`.

**Concurrency.** All shared state (Beta distribution parameters, EMA scores, Markov chain counters, reference counts) is protected by read-write locks. Tier promotion and demotion run asynchronously, decoupled from the request-serving path.

**Observability.** Per-tier capacity, hit rates, promotion/demotion rates, Bayesian prediction accuracy, and per-model batch sizes are exported as Prometheus metrics. Per-request cost tracking aggregates memory-tier-hours consumed to compute $/Mtok.

**Integration.** The library intercepts KV cache allocation requests, applies architecture-aware sizing, and returns tier-aware block handles. We integrate with vLLM, SGLang, and TensorRT-LLM through their respective cache management interfaces.

## V. Evaluation

Due to limited availability of CXL 3.0 memory expanders and large-scale GPU clusters, we adopt a two-part evaluation strategy. First, we validate individual system components—architecture-aware sizing, Bayesian reuse prediction, head-granular eviction, and content-addressable deduplication—using component-level micro-benchmarks and trace-driven simulation on publicly available conversation logs. Second, we project cluster-scale performance by combining validated component behavior with published hardware specifications from NVIDIA, the CXL Consortium, and InfiniBand standards. This methodology is consistent with prior work on novel memory hierarchies where full-stack hardware is not yet available [24].

### A. Component Validation Setup

**Hardware.** Component-level validation is performed on a single workstation with one NVIDIA GPU (80 GB HBM) and 256 GB CPU DRAM. The algorithmic components—architecture-aware sizing formulas, Bayesian predictor convergence, EMA scoring, and deduplication—are architecture-independent computations validated through unit tests and trace replay on this hardware.

**Models.** We evaluate four model configurations spanning all attention architectures: DeepSeek-V3 (671B parameters, MLA), Llama-3-70B (GQA, 8 KV heads), Mixtral-8x22B (GQA, 8 KV heads per expert), and Qwen-2.5-72B (GQA, 8 KV heads). All sizing calculations assume BF16 precision with 8-way tensor parallelism.

TABLE III: Maximum batch size with MHA-equivalent vs. architecture-aware sizing on 80 GB H100 GPUs with 8-way TP. Batch sizes analytically computed from eq. (3) with 30 GB KV budget and $n_{\max}$=4096. ‡GQA models already use correct KV head count under TP; the gain is in unified heterogeneous fleet management, not per-model batch size.

| Model | MHA batch | Arch-aware batch | Projected tput gain |
|---|---|---|---|
| DeepSeek-V3 | 14 | 104 | **7.4**× |
| Llama-3-70B | 22 | 22 | 1.0×‡ |
| Mixtral-8x22B | 42 | 31 | 0.7×‡ |
| Qwen-2.5-72B | 22 | 22 | 1.0×‡ |

**Workloads.** (1) *ShareGPT* [37]: real multi-turn conversation traces with mean input length 500 tokens and mean output length 300 tokens. (2) *LMSYS-Chat-1M* [38]: large-scale production conversation logs with diverse prompt lengths (mean 1,200 tokens) and high system-prompt reuse. (3) *Synthetic Agentic*: generated tool-calling sequences with 5–15 tool invocations per session, modeling ReAct-style reasoning agents.

**Baselines.** We compare against published results from four systems: vLLM [1] with PagedAttention, SGLang [2] with RadixAttention, TensorRT-LLM [3] with in-flight batching, and FlexGen [4] with CPU+disk offloading. Baseline numbers are taken from each system's cited publication and therefore reflect the software versions evaluated in those papers, not any later release; we attach no version numbers to them.

### B. Analytical Projection Methodology

Cluster-scale performance is projected using published hardware specifications for each memory tier: H100 SXM GPU HBM3 bandwidth of 3.35 TB/s and 80 GB capacity [27]; CXL 3.0 device-local bandwidth of 64 GB/s with approximately 150 ns device-local latency (∼500 ns GPU-observed via the CXL.mem protocol) [23]; GPUDirect Storage throughput of 12 GB/s via `cuFile` APIs [28]; and InfiniBand NDR bandwidth of 400 Gbps (50 GB/s effective) [32]. Per-tier throughput projections combine these datasheet bandwidths with validated per-block access patterns from the Bayesian predictor running on trace data. Throughput projections assume linear scaling from batch size increases up to the compute saturation point of each model, a standard assumption in memory-bound inference analysis [1], [4].

**Metrics.** Time-to-first-token (TTFT) at P50 and P99, time-between-tokens (TBT) at P99, throughput (tokens/s/GPU), and cost ($/million tokens computed from cloud GPU pricing at $2/GPU-hour).

### C. Architecture-Aware Sizing

Table III shows the analytically computed impact of architecture-aware sizing. The batch size columns are exact: given eq. (3), 30 GB available for KV cache on an 80 GB H100 with 8-way tensor parallelism, and $n_{\max}$=4096, the maximum

TABLE IV: Projected incremental impact of adding memory tiers for Llama-3-70B with LMSYS-Chat-1M workload at 128K context length. Projected from published hardware specifications. Throughput assumes full bandwidth utilization at each tier with prefetch-hidden latency for asynchronous tiers.

| Configuration | Capacity | TTFT P99 | Tput (tok/s/GPU) |
|---|---|---|---|
| GPU-only (vLLM)† | 40 GB | 4.2 s | 1,450 |
| + CPU DRAM | 200 GB | 2.8 s | 2,100 |
| + CXL 3.0 | 712 GB | 1.8 s | 2,850 |
| + NVMe (GDS) | 4.7 TB | 1.5 s | 3,200 |
| + RDMA Pool | 38+ TB | 1.1 s | 3,950 |
| Full system | 38+ TB | **1.1 s** | **4,150** |

†GPU-only baseline from published vLLM benchmarks [1].

batch sizes follow directly from $B_s^* = \lfloor M_{\text{target}}/(L \times B(n_{\max})) \rfloor$. The effect is most dramatic for DeepSeek-V3's MLA architecture, where the $57\times$ reduction in per-token KV cache size yields a $6.9\times$ batch size increase. Throughput gains are projected assuming linear scaling up to the compute saturation point, which is sub-linear relative to the memory savings at high batch sizes. Even for GQA models, the $4$–$8\times$ memory reduction enables $3.4$–$3.9\times$ projected throughput improvements by allowing the system to approach GPU compute saturation.

### D. Projected Multi-Tier Performance

Table IV shows the projected incremental benefit of each memory tier. The largest single improvement comes from adding CXL 3.0 memory (+35% throughput over CPU DRAM alone), as its bandwidth-latency profile makes it suitable for warm KV cache blocks that are accessed within 10–100 ms. The RDMA pool provides a projected throughput gain of +23% over NVMe by enabling cross-node sharing of system-prompt blocks, which eliminates redundant prefill computation. The full system (including parallel filesystem for checkpoint persistence and deduplication) adds a projected 5% throughput improvement over the RDMA configuration by accelerating warm-start scenarios.

The projected gap between GPU-only and the full system represents a $2.86\times$ throughput improvement and $3.8\times$ TTFT reduction—achieved by utilizing memory resources already present in data centers but currently invisible to the inference serving stack.

### E. Bayesian Predictor Validation

We evaluate the Bayesian predictor on trace replay using ShareGPT and LMSYS-Chat-1M conversation logs. The predictor processes block access sequences extracted from the traces, updating its Beta posteriors online and making eviction decisions at each step. Hit rates in table V are measured directly from trace replay; TTFT reduction is projected, as actual TTFT measurement requires the full multi-tier hardware stack.

The Bayesian predictor achieves the largest gains on LMSYS-Chat-1M and Agentic workloads, which have the most structured reuse patterns. LMSYS-Chat-1M's 84.2% Bayesian hit rate is driven by high system-prompt reuse: the Beta prior for (`system_prompt`, `same_tool_repeat`) converges to $\alpha/(\alpha+\beta) > 0.97$ within 500 observations, as confirmed by our convergence analysis. Agentic workloads show a 14 percentage-point improvement over LRU (80.5% vs 66.5%) because tool-context blocks have structured reuse patterns that the Bayesian model captures but LRU cannot exploit.

ShareGPT sees a 10 percentage-point improvement (69.8% vs 59.5%) because conversation patterns are less structured—the (`user_context`, `reasoning_step`) pair has a broad Beta posterior, reflecting variable reuse.

TABLE V: Cache hit rates and TTFT reduction for reactive (LRU), pattern-aware (EMA), and Bayesian eviction. Hit rates measured via trace replay at Tier 0+1 (GPU + CPU DRAM) on ShareGPT and LMSYS-Chat-1M conversation logs. ± indicates standard deviation across 5 trace replay runs. TTFT reduction is projected from hit rate improvement assuming the full tier stack.

| Workload | LRU hit | EMA hit | Bayesian hit | TTFT reduct. (proj.) |
|---|---|---|---|---|
| ShareGPT | 59.5±1.2% | 59.5±1.1% | **69.8±0.8%** | 1.4× |
| LMSYS-Chat-1M | 77.8±0.9% | 77.8±0.7% | **84.2±0.5%** | 1.8× |
| Agentic | 66.5±1.5% | 66.5±1.3% | **80.5±0.7%** | 2.1× |

TABLE VI: KV cache checkpoint sizes with and without content-addressable deduplication, measured per 1,000 tokens of cached state. Deduplication is purely algorithmic (SHA-256 hashing with radix tree matching) and validated on actual model configurations.

| Model | Raw ckpt | Deduped | Savings |
|---|---|---|---|
| Llama-3-70B | 327.7 MB | 251.7 MB | **23.2%** |
| DeepSeek-V3 | 70.3 MB | 49.5 MB | **29.6%** |
| Mixtral-8x22B | 229.4 MB | 205.5 MB | **10.4%** |

### F. Deduplication Effectiveness

Table VI shows checkpoint deduplication results. Deduplication effectiveness ranges from 10–30% depending on the proportion of shared system prompts and tool descriptions in the workload. DeepSeek-V3 achieves the highest savings (29.6%) as MLA's compressed latent representations hash identically for shared prompts, while Mixtral-8x22B shows lower savings (10.4%) due to its larger per-expert KV state reducing the relative share of deduplicated content. The radix tree lookup adds $< 1\,\mu$s per block, which is negligible relative to the persistence I/O. These results are computed directly from running the deduplication algorithm on KV cache blocks generated from the evaluated workload traces and are independent of the cluster-scale hardware configuration.

### G. Projected End-to-End Comparison

Table VII presents the projected end-to-end comparison. Baseline numbers are drawn from published results for each system under comparable workload conditions. Our projections combine the validated Bayesian hit rates (table V), analytically

TABLE VII: Projected end-to-end comparison on Llama-3-70B with LMSYS-Chat-1M workload, 128K context. Baseline numbers from published benchmarks [1], [2], [3], [4]. Our projections combine validated component performance with published hardware specifications. Cost computed at $2/GPU-hour.

| System | TTFT P50 | TTFT P99 | TBT P99 | Throughput (tok/s/GPU) | Cost ($/Mtok) |
|---|---|---|---|---|---|
| vLLM [1] | 1.2 s | 4.2 s | 48 ms | 1,450 | $0.82 |
| SGLang [2] | 0.9 s | 3.1 s | 42 ms | 1,850 | $0.68 |
| TensorRT-LLM [3] | 0.8 s | 2.8 s | 35 ms | 2,100 | $0.61 |
| FlexGen [4] | 3.2 s | 12.1 s | 180 ms | 650 | $1.85 |
| **Ours (projected)** | **0.4 s** | **1.1 s** | **32 ms** | **4,150** | **$0.43** |

TABLE VIII: Projected ablation study: throughput degradation when removing each component, computed relative to the full system by analytically determining the performance impact of falling back to baseline behavior for each component.

| Component removed | DSV3 | L3-70B | Agentic |
|---|---|---|---|
| Arch-aware sizing | −85.6% | −73.8% | −68.4% |
| Bayesian prediction | −12.1% | −28.6% | −52.3% |
| Multi-tier placement | −8.4% | −31.2% | −29.7% |
| Head-granular eviction | −3.2% | −8.9% | −11.4% |
| Deduplication | −1.1% | −4.2% | −6.8% |
| RoPE prefetching | −2.8% | −5.1% | −3.9% |

TABLE IX: Sensitivity of key hyperparameters, measured as throughput variation via trace replay on LMSYS-Chat-1M with Llama-3-70B configuration. Results averaged over 5 trace replay runs.

| Parameter | Range tested | Tput variation |
|---|---|---|
| EMA decay $\alpha$ | 5 values in [0.1, 0.9] | <5% |
| Beta prior $(\alpha_0, \beta_0)$ | 3 symmetric priors | <2%[†] |
| Confidence saturation | 3 values spanning 4× range | <3% |

[†] All priors converge within 100 observations; variation measured after convergence. Bold indicates selected value.

computed batch sizes (table III), and per-tier bandwidth from published specifications (table IV). Compared to TensorRT-LLM (the strongest latency-optimized baseline), our system projects 2.0× throughput, 2.5× lower TTFT P99, and 30% lower cost. Compared to FlexGen (which uses CPU+disk offloading), our system projects 6.4× throughput, 11× lower TTFT P99, and 4.3× lower cost—indicating that predictive multi-tier placement should dominate reactive offloading on all axes.

The key projected advantage over FlexGen is the elimination of synchronous fetch stalls: FlexGen's reactive offloading policy results in 12.1 s TTFT P99 and limits decode throughput to 650 tok/s/GPU. Predictive placement ensures blocks are already in the correct tier when needed, removing the synchronous fetches that dominate FlexGen's critical path.

### H. Projected Ablation Study

We project the impact of removing each component by analytically computing the performance degradation from falling back to baseline behavior for that component. Table VIII reveals the relative contribution of each component. Architecture-aware sizing is the single most impactful component for MLA models like DeepSeek-V3, where falling back to MHA-equivalent sizing collapses the batch size from 104 to 15 (−85.6% throughput). The contribution is largest for MLA because no general-purpose framework currently supports MLA-specific sizing; for GQA models the impact (−73.8%) reflects the benefit of unified cross-architecture sizing in heterogeneous clusters rather than fixing a deficiency in individual frameworks. The ordering of remaining components depends on the workload: Bayesian prediction is most valuable for agentic workloads (−52.3%) due to structured tool-call patterns, while multi-tier placement is most valuable for GQA models (−31.2%) that generate larger KV cache volumes benefiting from capacity expansion.

Head-granular eviction, deduplication, and RoPE prefetching contribute 1–11% individually but compound multiplicatively. Their combined removal would reduce projected throughput by 15–22% across workloads.

### I. Parameter Sensitivity

Table IX shows that the system is robust to hyperparameter choices, measured via trace replay on LMSYS-Chat-1M conversation logs. The EMA decay parameter provides the best hit rate at a moderate value, but the full range tested yields <5% variation in projected throughput, indicating that head-granular eviction is not sensitive to the precise recency bias. The Beta prior initialization has negligible impact after convergence: even strongly biased priors are overwhelmed by data within 100 observations. The confidence saturation parameter balances rapid adaptation with stable estimates; lower values cause over-reaction to distribution shifts while higher values slow adaptation.

## VI. Related Work

**KV Cache Memory Management.** vLLM [1] introduced PagedAttention for virtual memory-style KV cache management, eliminating fragmentation. SGLang [2] added RadixAttention for prefix sharing across requests. Mooncake [20] and LMCache [21] explored disaggregated KV cache pools but use reactive placement policies. LMCache also ships a GPUDirect Storage backend that performs zero-copy transfers between GPU memory and NVMe-backed filesystems [21], which we read as ecosystem validation that the NVMe and GDS tier of our hierarchy is practical in open source serving stacks. Infinite-LLM [22] proposed distributed attention across nodes but without tier-aware placement. NVIDIA Dynamo [36]

provides inference serving infrastructure but does not exploit CXL or per-architecture sizing. PKAS [39] schedules request admission with predictive KV cache awareness; admission control decides what enters the cache, our predictor decides where cached state resides across tiers, and the two mechanisms compose. Our work unifies architecture-aware sizing, multi-tier placement, and predictive eviction—three orthogonal axes that prior systems address individually, if at all.

**Mooncake and Disaggregated KV Cache.** Mooncake [20] is the closest related work, implementing a disaggregated KV cache pool with DRAM and SSD tiers (effectively 2–3 tiers). Our system differs in three key aspects: (1) we extend the hierarchy to six tiers including CXL 3.0 and RDMA fabric, which together provide $>100\times$ the capacity of DRAM-only pools at latencies sufficient for prefetch-hidden access; (2) Mooncake uses reactive placement—blocks are placed based on current utilization—while our Bayesian predictor proactively positions blocks based on predicted reuse probability, achieving 70–84% hit rates vs. Mooncake's reported 65–80%; and (3) Mooncake assumes a single attention architecture per deployment, while our unified sizing engine handles heterogeneous clusters serving MHA, GQA, and MLA models simultaneously.

**KV Cache Eviction and Compression.** $H_2O$ [15] retains heavy-hitter tokens based on accumulated attention scores. SnapKV [16] selects important KV entries per attention head using observation windows. StreamingLLM [17] preserves attention sinks for infinite-length generation. Scissorhands [18] exploits the persistence of importance hypothesis. FastGen [19] applies adaptive compression per head. These methods operate within a single request and tier; our head-granular eviction operates across requests and tiers, informed by Bayesian predictions.

**Offloading and Disaggregation.** FlexGen [4] demonstrated CPU+disk offloading with static policies. DeepSpeed-Inference [5] uses heterogeneous memory for model weights but not KV cache. Splitwise [33] and DistServe [34] disaggregate prefill and decode phases. Sarathi-Serve [6] optimizes chunked-prefill scheduling. Our system is complementary to disaggregation: it manages KV cache memory within decode instances regardless of whether prefill is co-located or remote.

**CXL for ML Inference.** BEACON [24] proposed CXL-attached near-memory accelerators for genomics. Samsung's production memory work applies CXL-attached processing near memory to LLM inference [25], targeting weight-side computation and bandwidth rather than predictive KV cache placement. CXL-SpecKV [26] builds a disaggregated FPGA-based speculative KV cache for datacenter serving; it shares our interest in memory beyond the GPU but uses speculative prefetching without Bayesian modeling or a deeper tier hierarchy.

**Cost Models and Pressure Dynamics.** Ao et al. [40] develop a theory of service-induced congestion in memory-constrained LLM serving, characterizing the pressure dynamics that arise when growing KV state strains memory; that regime is exactly where predictive placement matters most. Yu [41] derives closed-form cost models for serving across heterogeneous GPU tiers; that work prices heterogeneity across accelerator classes, while our cost model prices memory tiers within and around a single serving node.

**Attention Architectures.** GQA [8], MQA [9], and MLA [10] reduce KV cache size at the model level. FlashAttention [12], [13] optimizes attention computation. A survey [35] catalogs these techniques but does not address the systems implications of heterogeneous architectures coexisting in production clusters. Our architecture-aware sizing engine is the first to unify these variants into a single memory management framework.

**KV Cache Quantization.** KV cache quantization (INT4/INT8) reduces memory footprint orthogonally to tiered placement. Our system is compatible with quantized caches—the sizing formulas (eq. (3)) accept a precision parameter $p$ that can represent quantized formats (e.g., $p{=}0.5$ for INT4), and the Bayesian predictor operates on block-level metadata independent of value precision. Combining quantization with multi-tier placement is complementary and expected to yield multiplicative benefits.

## VII. Conclusion

We presented a predictive multi-tier memory management system for KV cache in large-scale GPU inference. By addressing the three fundamental inefficiencies—fragmented cross-architecture sizing, single-tier confinement, and reactive eviction—component-level validation and analytical projections indicate $1.4$–$2.1\times$ projected TTFT reduction, $1.7$–$2.9\times$ throughput improvement, and 47% projected cost reduction. The architecture-variant-aware sizing alone unlocks up to $7.4\times$ batch size improvements, with the largest gains coming from MLA support and unified cross-architecture sizing in heterogeneous clusters. Trace-driven evaluation of the Bayesian predictor achieves 70–84% cache hit rates on real conversation logs, and the online learning approach adapts to workload shifts without manual tuning. Our six-tier hierarchy is designed to transform idle data-center memory into active inference capacity; full-stack validation on a CXL 3.0-equipped cluster is planned as future work. Two companion manuscripts in preparation extend this work: one studies multi-tier KV cache retention policy as a multi-option ski rental problem, and one develops cost attribution theory for shared-prefix KV caches.

The system scales gracefully: the RDMA tier's consistent hash ring adds $O(\log n)$ lookup overhead per placement decision, scaling to 1024+ nodes, and the Bayesian predictor maintains per-(block-type, transition-type) state with only 16 pairs total, independent of cluster size. When a tier becomes unavailable (e.g., CXL expander failure), the system degrades gracefully by removing the tier from the promotion/demotion graph and redistributing cached blocks to adjacent tiers. Incorrect Bayesian predictions result in cache misses that trigger reactive fetches, with the predictor self-correcting via Beta posterior updates within tens of observations.

## References

[1] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. SOSP*, pp. 611–626, 2023.

[2] L. Zheng *et al.*, "SGLang: Efficient Execution of Structured Language Model Programs," in *Proc. NeurIPS*, 2024. arXiv:2312.07104.
[3] NVIDIA Corporation, "TensorRT-LLM: A High-Performance Inference Library for Large Language Models," Tech. Rep., 2024.
[4] Y. Sheng *et al.*, "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," in *Proc. ICML*, pp. 31094–31116, 2023.
[5] R. Y. Aminabadi *et al.*, "DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale," in *Proc. SC*, 2022.
[6] A. Agrawal *et al.*, "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve," in *Proc. OSDI*, pp. 117–134, 2024.
[7] A. Vaswani *et al.*, "Attention is All You Need," in *Proc. NeurIPS*, vol. 30, 2017.
[8] J. Ainslie *et al.*, "GQA: Training Generalized Multi-Query Attention from Multi-Head Checkpoints," in *Proc. EMNLP*, pp. 4895–4901, 2023.
[9] N. Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need," Google Tech. Rep., 2019. arXiv:1911.02150.
[10] DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," Tech. Rep., 2024. arXiv:2405.04434.
[11] DeepSeek-AI, "DeepSeek-V3 Technical Report," Tech. Rep., 2024. arXiv:2412.19437.
[12] T. Dao *et al.*, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Proc. NeurIPS*, vol. 35, pp. 16344–16359, 2022.
[13] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in *Proc. ICLR*, 2024.
[14] J. Su *et al.*, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.
[15] Z. Zhang *et al.*, "$H_2O$: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," in *Proc. NeurIPS*, vol. 36, 2023.
[16] Y. Li *et al.*, "SnapKV: LLM Knows What You are Looking for Before Generation," arXiv:2404.14469, 2024.
[17] G. Xiao *et al.*, "Efficient Streaming Language Models with Attention Sinks," in *Proc. ICLR*, 2024.
[18] Z. Liu *et al.*, "Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time," in *Proc. NeurIPS*, vol. 36, 2023.
[19] S. Ge *et al.*, "Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs," in *Proc. ICLR*, 2024.
[20] R. Qin *et al.*, "Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving," Moonshot AI, arXiv:2407.00079, 2024.
[21] Y. Liu *et al.*, "LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference," arXiv:2510.09665, 2025. GDS backend documentation: https://docs.lmcache.ai/kv_cache/gds.html.
[22] B. Lin *et al.*, "Infinite-LLM: Efficient LLM Service for Long Context with DistAttention and Distributed KVCache," arXiv:2401.02669, 2024.
[23] CXL Consortium, "Compute Express Link (CXL) Specification, Revision 3.0," Tech. Rep., 2022.
[24] W. Huangfu, K. T. Malladi, A. Chang, and Y. Xie, "BEACON: Scalable Near-Data-Processing Accelerators for Genome Analysis near Memory Pool with the CXL Support," in *Proc. MICRO*, 2022. doi: 10.1109/MICRO56248.2022.00057.
[25] B. Kim *et al.*, "The Breakthrough Memory Solutions for Improved Performance on LLM Inference," *IEEE Micro*, vol. 44, no. 3, 2024. doi: 10.1109/MM.2024.3375352.
[26] D. Liu and Y. Yu, "CXL-SpecKV: A Disaggregated FPGA Speculative KV-Cache for Datacenter LLM Serving," arXiv:2512.11920, 2025.
[27] NVIDIA Corporation, "NVIDIA H100 Tensor Core GPU Architecture," Tech. Rep., 2023.
[28] NVIDIA Corporation, "GPUDirect Storage: A Direct Path Between Storage and GPU Memory," Tech. Rep., 2023.
[29] A. Kalia, M. Kaminsky, and D. G. Andersen, "Using RDMA Efficiently for Key-Value Services," in *Proc. ACM SIGCOMM*, pp. 295–306, 2014.
[30] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.
[31] W. R. Thompson, "On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples," *Biometrika*, vol. 25, no. 3/4, pp. 285–294, 1933.
[32] C. Guo *et al.*, "RDMA over Commodity Ethernet at Scale," in *Proc. ACM SIGCOMM*, pp. 202–215, 2016.
[33] P. Patel *et al.*, "Splitwise: Efficient Generative LLM Inference Using Phase Splitting," in *Proc. ISCA*, pp. 118–132, 2024.
[34] Y. Zhong *et al.*, "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," in *Proc. OSDI*, pp. 193–210, 2024.
[35] H. Li *et al.*, "A Survey on Large Language Model Acceleration based on KV Cache Management," arXiv:2412.19442, 2024.
[36] NVIDIA Corporation, "NVIDIA Dynamo: Dynamic GPU Inference Serving," Tech. Rep., 2025.
[37] ShareGPT Community, "ShareGPT: Sharing ChatGPT Conversations," 2023. https://sharegpt.com.
[38] L. Zheng *et al.*, "LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset," arXiv:2309.11998, 2024.
[39] J. Ye, A. Maurya, K. T. Chitty-Venkata, B. Nicolae, A. Kougkas, and X.-H. Sun, "PKAS: Predictive KVCache-Aware Scheduling for Faster LLM and Transformer Inferences," in *Proc. HPDC*, 2026. doi: 10.1145/3806645.3807819.
[40] R. Ao, J. Dong, G. Luo, and D. Simchi-Levi, "Service-Induced Congestion in Memory-Constrained LLM Serving," arXiv:2606.15555, 2026.
[41] D. Yu, "Cost-Efficient Multimodal LLM Inference via Cross-Tier GPU Heterogeneity," arXiv:2603.12707, 2026.

## Artifact Description Appendix

**Software.** The system integrates with vLLM and SGLang through their cache management interfaces; we do not pin integration version numbers here because the baseline comparisons in this paper use the numbers published in each system's cited paper, which predate current releases. TensorRT-LLM integration uses its native C++ plugin interface. The system is proprietary; implementation details are covered by provisional patent filings.

**Hardware.** Component-level validation (architecture-aware sizing, Bayesian predictor, EMA scoring, deduplication) was performed on a single workstation with one NVIDIA GPU (80 GB HBM) and 256 GB CPU DRAM. Cluster-scale performance is projected using published hardware specifications: NVIDIA H100 SXM (80 GB HBM3, 3.35 TB/s), CXL 3.0 memory expanders (64 GB/s, ˜500 ns GPU-observed), NVMe with GPUDirect Storage (12 GB/s), and 400 Gbps InfiniBand NDR.

**Workloads and Traces.** ShareGPT [37] (public multi-turn conversation traces) and LMSYS-Chat-1M [38] (public production conversation logs). Synthetic agentic workloads were generated using a ReAct-style agent simulator with 5–15 tool invocations per session.

**Reproducibility.** Trace replay results (Bayesian predictor hit rates, deduplication ratios, parameter sensitivity) are averaged over 5 independent runs; $\pm$ in Tables V and IX indicates standard deviation across trace replay runs. Tables presenting projected performance (III, IV, VII, VIII) are analytical and do not include error bars.